Resumen de la semana 2

Del AVR al ESP32 · Tipos de datos, tiempo y pulsadores

1. Del AVR al ESP32: el mismo principio

El ESP32 usa palabras de 32 bits y tiene muchos más recursos que el ATmega328P, pero se mantiene una idea central: **los pines se controlan con bits de registros de hardware**. La diferencia es que en el ESP32 muchas señales pueden asignarse a distintos pines mediante la GPIO matrix.

Tarea	AVR	ESP32
Elegir entrada o salida	`DDRB`	`GPIO_ENABLE_REG`
Poner una salida en 1	`PORTB`	`GPIO_OUT_W1TS_REG`
Poner una salida en 0	`PORTB`	`GPIO_OUT_W1TC_REG`
Leer una entrada	`PINB`	`GPIO_IN_REG`

Los registros **W1TS** (set) y **W1TC** (clear) permiten poner o quitar bits escribiendo 1 en la máscara. Es una única escritura y se evita el problema de leer, modificar y reescribir un registro mientras una interrupción u otro núcleo cambia algún bit.

```
#include "soc/gpio_reg.h"
constexpr uint8_t PIN_LED = 16;
constexpr uint32_t MASK_LED = 1UL << PIN_LED;

void setup() { pinMode(PIN_LED, OUTPUT); }
void loop() {
  REG_WRITE(GPIO_OUT_W1TS_REG, MASK_LED); // LED ON
  delay(500);
  REG_WRITE(GPIO_OUT_W1TC_REG, MASK_LED); // LED OFF
  delay(500);
}
```

El ejemplo de clase conecta un LED al **GPIO16** con resistencia de 330 Ω hacia GND. `pinMode()` realiza la configuración; las escrituras a registros controlan la salida.

2. Elegir bien los pines y cuidar el voltaje

En el ESP32 DevKit V1 de 30 pines no debe confundirse el número **GPIO** con la posición física del conector. Siempre hay que revisar lo que está impreso en la placa.

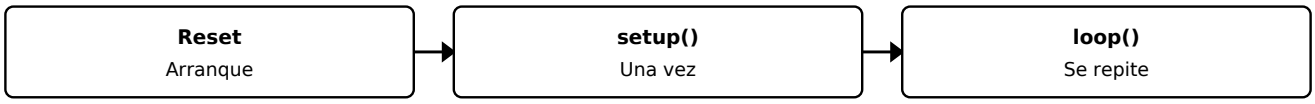
Grupo	Qué conviene recordar
GPIO 16, 17, 18, 19, 21	Adecuados para los ejemplos de LED y pulsador.
GPIO 34-39	Solo entrada; no tienen pull-up interna.
GPIO 0, 2, 5, 12, 15	Pines de arranque (strapping); usarlos con cuidado.
GPIO 1 y 3	UART0, usado al cargar y en el Monitor Serie.
GPIO 6-11	Conectados a la memoria flash; no usarlos.

Los GPIO del ESP32 trabajan a **3,3 V y no toleran 5 V**. Para un LED se necesita una resistencia que limite la corriente. Con LED rojo de caída aproximada de 2 V y resistencia de 330 Ω: `I = (3,3 − 2,0) / 330 ≈ 0,004 A`, es decir, **4 mA**.

> **Actividad de conexiones.** LED en GPIO18: adecuado. Pulsador en GPIO35 con `INPUT_PULLUP`: no, requiere resistencia externa. LED en GPIO2 o pulsador en GPIO12: mejor evitarlos por el arranque. Sensor de 5 V directamente al GPIO4: no es seguro. LED en GPIO1: interfiere con UART0.

3. Cómo arranca y se mantiene el programa

Tras encender o reiniciar el ESP32 se ejecuta el código de arranque, se prepara el entorno del sistema y se ejecuta `setup()` una sola vez. Después `loop()` se repite continuamente. En el ESP32 esta estructura funciona dentro de una tarea de FreeRTOS.



Hay tres formas de conservar variables: una **global** mantiene su valor, una **local normal** se crea otra vez al entrar a la función y una **local** `static` conserva el valor entre llamadas.

> **Predicción de `loop()`.** Si en cada vuelta se incrementan un contador global, un contador local inicializado en 0 y uno `static` inicializado en 0, en la **tercera vuelta** se imprime: `global=3 local=1 static=3`.

4. Tipos de datos y desbordamientos

En Arduino Uno `int` ocupa 2 bytes, mientras que en ESP32 ocupa 4 bytes. Para evitar depender de la placa conviene usar tipos con tamaño fijo: `uint8_t` (0 a 255), `int8_t` (−128 a 127), `uint16_t` y `uint32_t`.

Un entero **sin signo** da la vuelta cuando se supera su máximo. Por ejemplo, al incrementar `uint8_t c = 253` cinco veces se observan los valores `253, 254, 255, 0, 1`. El cálculo se hace con los bits disponibles y puede perder parte del resultado al guardarse.

```
uint8_t x = 200, y = 100;
uint8_t r = x + y;        // r termina en 44 (300 mod 256)
uint8_t a = 0xCC;
uint8_t inv = (uint8_t)~a; // 0x33
uint32_t m = 1UL << 31;   // mascara sin signo
```

`~a` puede promocionarse a `int` antes de invertir todos los bits, por eso se hace una conversión a `uint8_t` cuando interesa el resultado de 8 bits. `static` conserva el valor entre llamadas y `volatile` avisa al compilador de que una variable puede cambiar fuera del flujo normal; `volatile` **no significa operación atómica**.

Actividad: encontrar los errores

> **A.** `for (uint8_t i = 0; i < 256; i++)`**.** El contador vuelve a 0 al pasar de 255: el ciclo no termina. Se corrige usando `uint16_t i` o `int i`.
>
> **B.** `int t0 = millis();`**.** El tipo con signo no es adecuado para un tiempo creciente de 32 bits. Se usa `uint32_t t0 = millis();`.

C. `uint32_t m = 1 << 31;`**.** El 1 es de tipo `int`; se usa `1UL << 31` para desplazar un literal sin signo de 32 bits.

D. `int veces = 0; veces++;` **dentro de** `loop()`**.** Se reinicia en cada llamada; usar `static int veces = 0;` si debe recordar el total.

5. Medir tiempo sin detener el programa

`delay()` detiene el avance de `loop()` durante la espera y puede hacer que no se detecte una pulsación rápida. `millis()` permite revisar cuánto tiempo ha pasado **sin bloquear** otras tareas. La forma correcta compara tiempos por resta: `ahora - anterior >= PERIODO`.

```
constexpr uint32_t PERIODO_A = 500; // ms
constexpr uint32_t PERIODO_B = 130; // ms
uint32_t tA = 0, tB = 0;
bool estadoA = false, estadoB = false;

void loop() {
  uint32_t ahora = millis();
  if (ahora - tA >= PERIODO_A) {
    tA += PERIODO_A;
    estadoA = !estadoA;
    digitalWrite(16, estadoA);
  }
  if (ahora - tB >= PERIODO_B) {
    tB += PERIODO_B;
    estadoB = !estadoB;
    digitalWrite(17, estadoB);
  }
}
```

En ese ejemplo (adaptado de la diapositiva), se configuran previamente GPIO16 y GPIO17 como salidas. La resta funciona incluso si `millis()` desborda, siempre que se manejen los tiempos con tipos sin signo y el intervalo sea adecuado.

6. Pulsador: pull-up, rebote y flanco

Con `INPUT_PULLUP`, el botón conectado entre GPIO21 y GND se lee **HIGH al estar suelto** y **LOW al presionarlo**. El contacto mecánico produce varios cambios breves (rebote), así que se espera alrededor de **30 ms de estabilidad** antes de aceptar el nuevo estado.

No se debe contar cada vuelta donde el botón está en LOW, porque una sola pulsación duraría muchas vueltas. Lo correcto es detectar el **flanco de bajada**, es decir, la transición de HIGH a LOW.

```
constexpr uint8_t PIN_BTN = 21;
constexpr uint32_t T_REBOTE = 30;
bool lecturaAnterior = HIGH, estadoEstable = HIGH;
uint32_t tUltimoCambio = 0;
uint16_t pulsaciones = 0;

void loop() {
  bool lectura = digitalRead(PIN_BTN);
  if (lectura != lecturaAnterior) {
    tUltimoCambio = millis();
    lecturaAnterior = lectura;
  }
  if (millis() - tUltimoCambio >= T_REBOTE &&
      lectura != estadoEstable) {
    estadoEstable = lectura;
    if (estadoEstable == LOW) pulsaciones++;
  }
}
```

7. Aplicación en la AP2 y fallas frecuentes

La estructura del **juego de luces** combina todo lo anterior: se consulta el botón con antirrebote para cambiar la velocidad, se usa `millis()` para avanzar la posición de la luz y, al final, se actualizan los LED. Así la animación sigue mientras el botón permanece disponible.

```
void loop() {
  // 1. Entradas: boton con antirrebote
  if (botonPresionado())
    velocidad = (velocidad + 1) % N_VEL;
  // 2. Logica: avanzar por tiempo
  uint32_t ahora = millis();
  if (ahora - tPaso >= PERIODOS[velocidad]) {
    tPaso = ahora;
    posicion = (posicion + 1) % N_LEDS;
  }
  // 3. Salidas: mostrar la posicion actual
  for (uint8_t i = 0; i < N_LEDS; i++)
    digitalWrite(LEDS[i], i == posicion);
}
```

Este es el esquema de organización presentado en clase. Las variables, funciones y arreglos usados deben estar declarados en el programa completo.

Problema al usar la placa	Primero revisar
No aparece puerto	Cable USB de datos y controlador CP210x/CH340.
No sube el programa	Botón BOOT al entrar en modo carga.
Caracteres raros en monitor	La velocidad de serie (por ejemplo, 115200).
El ESP32 no arranca con el circuito	Pines de strapping o alimentación.

> **Idea que conecta la semana:** primero leer entradas, luego actualizar la lógica usando `millis()` y finalmente escribir las salidas. Eso permite que el juego de luces responda sin detenerse con `delay()`.